OVERTONE Manual

OVERTONE computes anharmonic vibrational spectra with second-order vibrational perturbation theory (VPT2). It works with any calculator of the Atomic Simulation Environment (ASE). It provides fundamentals, overtones, combination bands, IR intensities and plotted spectra.

This manual first explains the installation and a complete example with default settings. The reference part then describes every function and class a user can call, with all parameters, what they control and their defaults.

Contents

1. Installation
2. A complete example
3. Concepts and units
4. Workflow functions
5. Harmonic analysis
6. Quartic force field
7. Dipole derivatives
8. VPT2
9. Transitions
10. Spectra
11. Coordinate transformations
12. Engines
13. Files written by OVERTONE
14. Recommendations and troubleshooting

---

1. Installation

Requirements

- Python 3.10 or later
- NumPy and SciPy
- ASE 3.23 or later
- pydantic 2.0 or later
- rich (formatted tables)
- matplotlib (spectra)

These are installed automatically. In addition you need at least one ASE calculator that provides energies (and ideally forces and dipole moments).

Installation with pip

```
pip install overtone
```

or the latest version from GitHub

```
pip install git+https://github.com/thequantumchemist/overtone.git
```

Optional extras

Extra	Installs	Needed for
`cluster`	shephex	distributing the single points over a queueing system
`falcon`	falcon-md	on-the-fly active learning
`tutorial`	tblite	the tutorials (GFN2-xTB)
`test`	pytest, tblite	running the test suite

```
pip install "overtone[tutorial,cluster]"
```

Installation for development

```
git clone https://github.com/thequantumchemist/overtone.git
cd overtone
pip install -e ".[test]"
pytest
```

---

2. A complete example

The example computes the anharmonic spectrum of water with default settings. It uses the semi-empirical method GFN2-xTB through its ASE calculator from the tblite package (`pip install tblite`). Any other ASE calculator can be used in the same way. A list of supported calculators is given on the ASE calculators page.

```python
from ase.build import molecule
from ase.optimize import BFGS
from tblite.ase import TBLite

from overtone import calculate_anharmonics


def setup_calculator():
    """Returns a new calculator. OVERTONE calls it for every structure."""
    return TBLite(method="GFN2-xTB", verbosity=0, accuracy=0.01)


# 1. optimise the structure tightly
atoms = molecule("H2O")
atoms.calc = setup_calculator()
BFGS(atoms).run(fmax=1e-4)

# 2. harmonic analysis, quartic force field and VPT2 in one call
result = calculate_anharmonics(atoms, setup_calculator, "water")

# 3. results
result.summary()               # anharmonicity constants and fundamentals
result.transitions_table()     # fundamentals, overtones, combination bands
result.plot_spectrum()         # needs intensities=True for a spectrum, see below
```

With default settings OVERTONE uses GVPT2, energies, a five-point stencil and a step of 0.2 in dimensionless normal coordinates. Common additions are

```python
result = calculate_anharmonics(
    atoms, setup_calculator, "water",
    derivative_source="forces",   # fewer single points when forces are available
```

```
    intensities=True,             # IR intensities (calculator must provide dipoles)
    max_quanta=3,                 # also bands with three quanta
    save_results=True,            # JSON files for later analysis
)
result.plot_spectrum(log=True)    # log scale shows the weak near-IR bands
```

The `tutorials` folder contains further examples.

---

3. Concepts and units

Workflow

1. **Harmonic analysis.** The Hessian is computed from central differences of the forces. Diagonalisation gives harmonic frequencies and normal modes.
2. **Quartic force field.** The structure is displaced along the normal modes. Nested finite differences of the energies (or forces) give the cubic constants `phi_ijk` and the quartic constants `phi_iijj`.
3. **VPT2.** The anharmonicity constants `X_ij` give fundamentals, overtones and combination bands. Fermi resonances are detected and, with GVPT2, treated variationally.
4. **Intensities (optional).** Dipole derivatives from the same displaced structures give transition moments and IR intensities.

Units

Quantity	Unit
Positions, Cartesian steps	Å
Energies returned by ASE calculators	eV
Wavenumbers, force constants T, U, anharmonicity constants X	cm^{-1}
Dipole moments and their derivatives	e·Å (ASE convention)
IR intensities	km/mol
Anharmonic step `delta`	dimensionless (reduced normal coordinates)

The anharmonic step is dimensionless

The step for the quartic force field is given in dimensionless normal coordinates q, not in Ångström. The Cartesian displacement of each atom follows from the frequency of the mode and the atomic masses. Stiff modes and heavy atoms move less than soft modes and light atoms. For water, a step of 0.1 moves the atoms by 0.007 to 0.010 Å.

Numerical noise

Fourth derivatives amplify random errors of the energies as $1/\delta^4$. An energy error of 10^{-8} Hartree scatters the fundamentals of water by about 13 cm^{-1} at $\delta = 0.1$, but by less than 1 cm^{-1} at $\delta = 0.2$. Converge every single point tightly and keep the default step unless you have a reason to change it.

Mode indices

Modes are numbered from 0 in the order of increasing harmonic frequency. The first `n_tr` modes are translations and rotations (6, or 5 for linear molecules, 0 with constrained atoms). Band labels such as v7 or 2v7+v9 use the mode index plus one.

---

4. Workflow functions

`calculate_anharmonics`

```
calculate_anharmonics(atoms, setup_calculator, directory, engine=None,
                      setup_harmonic_calculator=None, constrain_index=None,
                      active_modes=None, exclude_below=0.0, harmonic_delta=0.005,
                      symmetrize_hessian=True, project_translations_rotations=True,
                      linear=None, derivative_source="energy", anharmonic_delta=0.2,
                      stencil=1, coupling=3, method="gvpt2", coriolis=True,
                      resonance_window=200.0, resonance_K1=1.0, resonance_K2=1.0,
                      min_frequency=20.0, max_quanta=2, intensities=False,
                      save_results=False, dry_run=False)
```

Runs the complete workflow and returns a VPT2Result. With `dry_run=True` it returns a dictionary with the number of single points instead.

Input

Parameter	Default	Controls
`atoms`	required	Equilibrium structure (ASE `Atoms`). Optimise it tightly with the same calculator (fmax ≈ 10^{-4} eV/Å).
`setup_calculator`	required	Function without arguments that returns a new ASE calculator. It is called for every structure.
`directory`	required	Working directory. The subfolders `harmonics/` and `anharmonics/` are created inside.
`engine`	None	How the single points are run. `None` uses `SerialEngine`. `ShephexEngine` distributes them over a queueing system.
`setup_harmonic_calculator`	None	Separate calculator for the harmonic analysis, e.g. a higher level of theory (hybrid scheme) or a calculator with numerical gradients. `None` uses `setup_calculator`.

System

Parameter	Default	Controls
`constrain_index`	None	Indices of atoms that stay fixed (e.g. a surface). Their coordinates are not displaced. Projection of translations/rotations and Coriolis terms are switched off. Fix at least three non-collinear atoms.
`active_modes`	None	Mode indices treated anharmonically. `None` uses all vibrational modes.
`exclude_below`	0.0	Modes with a harmonic frequency below this value (cm^{-1}) are excluded after the harmonic analysis, including imaginary modes. Useful for torsions. Ignored if `active_modes` is given.

Harmonic analysis

Parameter	Default	Controls
`harmonic_delta`	`0.005`	Cartesian step for the Hessian in Å. Smaller steps reduce the anharmonic truncation error but need tight SCF convergence.
`symmetrize_hessian`	`True`	Use $(H + H^T)/2$. With `False` the lower triangle of the raw Hessian is used.
`project_translations_rotations`	`True`	Project out translations and rotations (Eckart conditions). Their frequencies become exactly zero.
`linear`	`None`	Force linear (`True`) or non-linear (`False`) treatment. `None` detects linear molecules automatically.

Quartic force field

Parameter	Default	Controls
`derivative_source`	`"energy"`	`"energy"` differentiates energies (works with every calculator). `"forces"` differentiates analytic forces and needs far fewer single points.
`anharmonic_delta`	`0.2`	Step in dimensionless normal coordinates (not Å). See Numerical noise.
`stencil`	`1`	Finite-difference stencil. 0 = 2 points, 1 = 5 points, 2 = 7 points, 3 = 9 points per derivative. Higher stencils need many more single points.
`coupling`	`3`	3 includes all cubic constants `phi_ijk`. 2 includes only constants of type `phi_iij`, which is cheaper but misses three-mode couplings and type-2 Fermi resonances.

VPT2

Parameter	Default	Controls
`method`	`"gvpt2"`	`"gvpt2"` treats Fermi resonances variationally (identical to standard VPT2 without resonances). `"standard"` uses no resonance treatment. `"deperturbed"` removes resonant terms without variational step. `"dcpt2"` uses degeneracy-corrected perturbation theory. `"split"` reports cubic and quartic contributions separately.
`coriolis`	`True`	Include Coriolis contributions to X.
`resonance_window`	`200.0`	Largest frequency gap (cm^{-1}) for a Fermi resonance.
`resonance_K1`	`1.0`	Threshold (cm^{-1}) of the Martin test for type-1 resonances ($2\omega_j \approx \omega_i$).
`resonance_K2`	`1.0`	Threshold (cm^{-1}) of the Martin test for type-2 resonances ($\omega_j + \omega_k \approx \omega_i$).

Parameter	Default	Controls
`min_frequency`	`20.0`	Active modes below this frequency (cm^{-1}), or imaginary ones, stop the run right after the harmonic analysis. Modes below 100 cm^{-1} only give a warning.

Output

Parameter	Default	Controls
`max_quanta`	2	Highest total number of quanta in the band list. 2 gives overtones and binary combinations, 3 adds the second-overtone region.
`intensities`	`False`	Compute IR intensities. Requests dipole moments from the calculator. Keep `False` for calculators without dipoles.
`save_results`	`False`	Write all intermediate results as JSON files to `directory`. Needed for `recompute_vpt2`.
`dry_run`	`False`	Only print and return the number of single points. Nothing is calculated.

`estimate_cost`

```
estimate_cost(atoms, derivative_source="energy", stencil=1, coupling=3,
              active_modes=None, constrain_index=None, linear=None)
```

Returns the number of single points without running anything, as a dictionary with the keys `harmonic`, `anharmonic`, `total` and `n_modes`. The parameters have the same meaning as in `calculate_anharmonics`. Modes removed later by `exclude_below` are still counted, so the result is then an upper bound.

`recompute_vpt2`

```
recompute_vpt2(directory, method="gvpt2", intensities=True, **config_overrides)
```

Repeats only the VPT2 step of a finished run that used `save_results=True`. No new single points are calculated.

Parameter	Default	Controls
`directory`	required	Run directory given to `calculate_anharmonics`.
`method`	`"gvpt2"`	VPT2 variant, see `calculate_anharmonics`.
`intensities`	`True`	Use the saved dipole derivatives, if present.
`**config_overrides`		Any field of `VPT2Config`, e.g. `max_quanta=3`, `coriolis=False` or `resonance_K2=5.0`.

---

5. Harmonic analysis

`Harmonics`

```
Harmonics(atoms, delta=0.005, constrain_index=None, symmetrize_hessian=True,
          project_translations_rotations=True, linear=None, linear_tolerance=1e-05)
```

Parameter	Default	Controls
`atoms`	required	Equilibrium structure.
`delta`	`0.005`	Cartesian step for the central differences in Å.
`constrain_index`	`None`	Fixed atoms. Switches the projection off.
`symmetrize_hessian`	`True`	Use $(H + H^T)/2$.
`project_translations_rotations`	`True`	Eckart projection of the rigid-body motions.
`linear`	`None`	Override of the automatic linear-molecule detection.
`linear_tolerance`	`1e-05`	A molecule counts as linear if its smallest principal moment of inertia is below this fraction of the largest one.

Methods

Method	Description
`calculate_displacements()`	Returns the displaced structures: the reference, then `+delta` and `−delta` for every coordinate of every unconstrained atom.
`calculate_hessian(trajectory)`	Builds the Hessian from the forces of the structures returned by `calculate_displacements()` (same order, forces attached) and returns a `HarmonicsResult`.
`analyse_hessian(hessian)`	Normal modes and frequencies from a given Cartesian Hessian in eV/Å², e.g. from another program. Returns a `HarmonicsResult`.

`HarmonicsResult`

Field	Description
`hessian`	Raw Cartesian Hessian (eV/Å²).
`w2`	Eigenvalues of the mass-weighted Hessian.
`modes`	Normal modes in mass-weighted Cartesian coordinates, one per column.
`freq`	Harmonic wavenumbers (cm^{-1}), real part.
`ifreq`	Imaginary part (cm^{-1}). Non-zero for imaginary modes.
`freq_eV`	Frequencies in eV (complex).
`chemical_formula`	Chemical formula of the molecule.
`n_tr`	Number of rigid-body modes at the beginning of the mode list (default 6).
`is_linear`	Whether the molecule was treated as linear (default `False`).
`projected`	Whether translations/rotations were projected out (default `False`).
`symmetrized`	Whether the Hessian was symmetrized (default `False`).
`hessian_asymmetry`	$\max\|H - H^T\| / \max\|H\|$ of the raw Hessian, a measure of its numerical quality (default `0.0`).

Method or property	Description
`vibrational_modes`	Property. Indices of the vibrational modes.

Method or property	Description
`check_active_modes(active_modes, min_frequency=20.0, warn_frequency=100.0)`	Raises an error for imaginary modes or modes below `min_frequency`, warns below `warn_frequency` (cm^{-1}).
`get_real_frequencies()`	Real frequencies.
`get_imaginary_frequencies()`	Imaginary frequencies.
`has_imaginary_frequencies()`	True if any mode is imaginary.
`summary(show_all=True, file=None)`	Prints a table of the frequencies. `show_all=False` hides the rigid-body modes. `file` redirects the output.
`dump(filepath)` / `HarmonicsResult.load(filepath)`	Write to and read from JSON.
`is_equivalent(other, rtol=1e-05, atol=1e-08)`	Compares two results within tolerances.

`is_linear_molecule`

```
from overtone.harmonics.harmonics import is_linear_molecule
is_linear_molecule(atoms, tolerance=1e-05)
```

Returns True if the smallest principal moment of inertia is below `tolerance` times the largest one.

---

6. Quartic force field

`AnharmonicDerivativesConfig`

Settings for the displacements and finite differences. Usually created with `AnharmonicDerivativesConfig.from_harmonics_result(...)`.

```
AnharmonicDerivativesConfig.from_harmonics_result(
    harmonics_result, atoms, active_modes, delta=0.2, coupling=3, stencil=1,
    constrained=False, unconstrained_index=None, force_indices=None)
```

Field	Default	Controls
`freq`	required	Harmonic frequencies (cm^{-1}).
`modes`	required	Normal modes from the harmonic analysis.
`atoms`	required	Equilibrium structure.
`active_modes`	required	Modes to displace.
`delta`	`0.2`	Step in dimensionless normal coordinates.
`coupling`	3	2 or 3, see `calculate_anharmonics`.
`stencil`	1	0 to 3, see `calculate_anharmonics`.
`masses`	None	Atomic masses, filled automatically.
`constrained`	False	Whether some atoms are fixed.
`unconstrained_index`	None	Indices of the atoms that move (with `constrained=True`).
`force_indices`	None	Cartesian indices of the moving atoms (with `constrained=True`).
`len_modes`	None	Total number of modes, filled automatically.
`num_atoms`	None	Number of moving atoms, filled automatically.
`freq_array`	None	Internal copy of the frequencies, filled automatically.

`dump(filepath)` and `AnharmonicDerivativesConfig.load(filepath)` write and read JSON.

AnharmonicDerivativesSolver

```
AnharmonicDerivativesSolver(config)
```

Method	Description
`generate_displacement_trajectory()`	Displaced structures and their keys for the energy route. Returns `(trajectory, keys)`.
`generate_displacement_trajectory_forces()`	The same for the force route.
`calculate_derivatives(trajectory_with_energies, displacement_keys=None)`	Force constants from the energies of the trajectory. Returns an `AnharmonicDerivativesResult`.
`calculate_derivatives_from_forces(trajectory_with_forces, displacement_keys=None)`	Force constants from the forces of the trajectory.
`get_dipole_displacement_keys()`	Keys of the structures whose dipoles are needed for intensities.
`missing_dipole_keys(available_keys)`	Dipole keys that are not part of `available_keys`. Normally empty.
`calculate_dipole_derivatives(trajectory_with_dipoles, displacement_keys)`	Dipole derivatives from a trajectory with dipole moments. Returns a `DipoleDerivativesResult`.
`create_displaced_trajectory(displacement_keys)`	Structures for arbitrary displacement keys.
`save_trajectory(trajectory, filename)`	Writes a trajectory to a file.

AnharmonicDerivativesResult

Field	Description
T	Cubic force constants `phi_ijk` (cm^{-1}), shape (n, n, n).
U	Quartic force constants `phi_iijj` (cm^{-1}), shape (n, n, n, n).
V	Energies of the displaced structures, keyed by displacement.
`required_calculations`	Displacement keys that were calculated.
`number_of_calculations`	Number of single points.

Method	Description
`summary()`	Prints statistics of the force constants.
`get_T_statistics()` / `get_U_statistics()`	Statistics of T and U as dictionaries.
`get_nonzero_T_count()` / `get_nonzero_U_count()`	Number of non-zero constants.
`dump(filepath)` / `AnharmonicDerivativesResult.load(filepath)`	Write to and read from JSON.
`is_equivalent(other, rtol=1e-05, atol=1e-08)`	Compares two results.

---

7. Dipole derivatives

DipoleDerivativesCalculator

```
DipoleDerivativesCalculator(n_modes, active_modes, delta, stencil=1)
```

Parameter	Default	Controls
`n_modes`	required	Total number of modes.
`active_modes`	required	Modes for which derivatives are needed.
`delta`	required	Step in dimensionless normal coordinates (same as the force field).
`stencil`	1	Finite-difference stencil.

Method	Description
`get_displacement_keys()`	Displacement keys at which dipoles are needed.
`calculate(dipoles)`	Derivatives from a dictionary `{key: dipole vector in e·Å}`. Returns a `DipoleDerivativesResult`.

`DipoleDerivativesResult`

Field	Description
`mu1`	First derivatives dμ/dq_i (e·Å), shape (n_modes, 3).
`mu2`	Second derivatives d²μ/dq_i dq_j (e·Å), shape (n_modes, n_modes, 3).

`dump(filepath)` and `DipoleDerivativesResult.load(filepath)` write and read JSON.

---

8. VPT2

`VPT2Config`

Usually created with `VPT2Config.from_harmonics_result(...)`.

```
VPT2Config.from_harmonics_result(
    harmonics_result, active_modes, atoms=None, coupling=3, method="gvpt2",
    max_quanta=2, coriolis=True, resonance_window=200.0,
    resonance_K1=1.0, resonance_K2=1.0)
```

Field	Default	Controls
`freq`	required	Harmonic frequencies (cm^{-1}).
`active_modes`	required	Modes treated anharmonically.
`min_mode_idx`	required	Lowest active mode index.
`max_mode_idx`	required	Highest active mode index.
`len_modes`	required	Total number of modes.
`coupling`	3	Coupling level of the force field.
`masses`	None	Atomic masses.
`modes`	None	Normal modes (needed for Coriolis terms).
`atoms`	None	Equilibrium structure (needed for Coriolis terms).
`freq_array`	None	Internal copy of the frequencies.
`method`	`"gvpt2"`	VPT2 variant. The aliases `"vpt2"`, `"dvpt2"`, `"generalized"` and `"degeneracy_corrected"` are accepted.
`max_quanta`	2	Highest total number of quanta in the band list.

Field	Default	Controls
`coriolis`	`True`	Include Coriolis contributions.
`resonance_window`	`200.0`	Largest frequency gap for a Fermi resonance (cm⁻¹).
`resonance_K1`	`1.0`	Martin-test threshold for type-1 resonances (cm⁻¹).
`resonance_K2`	`1.0`	Martin-test threshold for type-2 resonances (cm⁻¹).

`dump(filepath)` and `VPT2Config.load(filepath)` write and read JSON.

VPT2Solver

```
VPT2Solver(config, derivatives, dipole_derivatives=None)
```

Parameter	Default	Controls
`config`	required	A `VPT2Config`.
`derivatives`	required	An `AnharmonicDerivativesResult` or a tuple `(T, U)` of force-constant arrays in cm⁻¹.
`dipole_derivatives`	`None`	A `DipoleDerivativesResult`. Without it, bands are listed without intensities.

Method	Description
`calculate()`	Runs the method chosen in the configuration and returns a `VPT2Result` with all transitions.
`calculate_anharmonic(add_coriolis=True)`	Standard VPT2.
`calculate_anharmonic_split()`	Standard VPT2 with cubic and quartic contributions reported separately.
`calculate_anharmonic_deperturbed(add_coriolis=True, threshold=None, K1=None, K2=None)`	Deperturbed VPT2. `None` uses the resonance settings of the configuration.
`calculate_anharmonic_degeneracy_corrected(add_coriolis=True)`	DCPT2.
`calculate_transitions(X, max_quanta=2, variational=False)`	Band list from an X matrix. `variational=True` applies GVPT2.
`resonance_check(threshold=None, K1=None, K2=None)`	Detected Fermi resonances as two lists, type 1 as `(i, j)` with ω_i ≈ 2ω_j and type 2 as `(i, j, k)` with ω_i ≈ ω_j + ω_k.
`calculate_fundamental_frequencies(X)`	Fundamentals from an X matrix.
`calculate_anharmonic_coriolis()`	Coriolis contributions to X.
`calculate_zeta()`	Coriolis constants in the principal axis frame, shape (3, n, n).
`calculate_rotational_constants()`	Rotational constants B_e (cm⁻¹).
`calculate_inertia()`	Principal moments of inertia (amu·Å²) as a diagonal matrix.

VPT2Result

Field	Description
X	Anharmonicity constants (cm⁻¹). For deperturbed VPT2 and GVPT2 the deperturbed constants.

Field	Description
`anharm_freq`	Fundamentals (cm⁻¹), indexed like the modes. For GVPT2 after the variational treatment.
`method`	VPT2 variant used.
`X_3`	Cubic contributions to X (`split` only, default `None`).
`X_4`	Quartic contributions to X (`split` only, default `None`).
`X_coriolis`	Coriolis contributions to X (default `None`).
`zeta`	Coriolis constants (default `None`).
`fermi_type1`	Detected type-1 resonances (default `None`).
`fermi_type2`	Detected type-2 resonances (default `None`).
`transitions`	List of `VPT2Transition`, sorted by energy (default `None`).

Method or property	Description
`summary(active_modes=None, file=None)`	Prints anharmonicity constants and fundamentals.
`transitions_table(kinds=None, xmin=None, xmax=None, file=None)`	Prints the band list. `kinds` selects `"fundamental"`, `"overtone"` and/or `"combination"`. `xmin` and `xmax` limit the wavenumber range.
`get_transitions(kinds=None, xmin=None, xmax=None)`	Returns the selected bands as a list.
`has_intensities`	Property. `True` if intensities were computed.
`spectrum(**kwargs)`	Broadened spectrum as arrays `(x, y)`. Accepts the parameters of `broaden_transitions`.
`plot_spectrum(**kwargs)`	Plots the spectrum. Accepts the parameters of `plot_spectrum`.
`get_active_X(active_modes)`	X matrix of the given modes.
`get_active_frequencies(active_modes)`	Fundamentals of the given modes.
`dump(filepath)` / `VPT2Result.load(filepath)`	Write to and read from JSON.
`is_equivalent(other, rtol=1e-05, atol=1e-08)`	Compares two results.

---

9. Transitions

`VPT2Transition`

One band, i.e. a transition from the vibrational ground state.

Field	Default	Description
`quanta`	required	Excited state as `{mode index: number of quanta}`.
`kind`	required	`"fundamental"`, `"overtone"` or `"combination"`.
`label`	required	Band label, e.g. `"2v7"` or `"v7+v9"` (mode index plus one).
`harmonic`	required	Harmonic wavenumber (cm⁻¹).
`energy`	required	Anharmonic wavenumber (cm⁻¹).
`intensity`	None	IR intensity (km/mol). `None` without dipoles and for three or more quanta.

Field	Default	Description
`transition_dipole`	`None`	Transition dipole moment (e·Å), as `[x, y, z]`.
`resonance_affected`	`False`	`True` if the band is part of a detected Fermi resonance.
`mixing`	`None`	GVPT2 only. Weights of the zeroth-order states, e.g. `{"v11": 0.71, "v8+v9": 0.29}`.

The property `total_quanta` gives the total number of quanta.

`TransitionCalculator`

Computes the band list from an X matrix. Used internally by `VPT2Solver`.

```
TransitionCalculator(freq, X, active_modes, T=None, mu1=None, mu2=None,
                     fermi_type1=(), fermi_type2=())
```

Parameter	Default	Controls
`freq`	required	Harmonic wavenumbers (cm^{-1}).
`X`	required	Anharmonicity constants (cm^{-1}).
`active_modes`	required	Active modes.
`T`	`None`	Cubic force constants. Needed for intensities and GVPT2.
`mu1`	`None`	First dipole derivatives. Needed for intensities.
`mu2`	`None`	Second dipole derivatives. Needed for intensities.
`fermi_type1`	`()`	Type-1 resonances from `resonance_check`.
`fermi_type2`	`()`	Type-2 resonances from `resonance_check`.

Method	Description
`calculate(max_quanta=2, variational=False)`	Band list. `variational=True` applies GVPT2.
`transition_moment(quanta)`	Transition dipole of a band, or `None` beyond the implemented order.
`is_resonance_affected(quanta)`	Whether a band takes part in a detected resonance.

`term_value` and `enumerate_states`

```
from overtone.anharmonics.transitions import term_value, enumerate_states
term_value(freq, X, quanta, active_modes)
enumerate_states(active_modes, max_quanta)
```

`term_value` returns the VPT2 term value of the state `quanta` relative to the ground state (cm^{-1}). `enumerate_states` lists all excited states with up to `max_quanta` quanta. The constant `INTENSITY_CONSTANT` (57.83) converts $\tilde{\nu}|M|^2$ (cm^{-1}, $e^2Å^2$) to km/mol.

---

10. Spectra

`plot_spectrum`

```
from overtone import plot_spectrum
plot_spectrum(result, shape="lorentzian", fwhm=None, kinds=None, xmin=None,
              xmax=None, resolution=0.5, sticks=True, by_kind=True, log=False,
              normalize=False, show_harmonic=False, x_unit="cm-1", invert_x=True,
              ax=None, label=None, color="black", reference=None,
              reference_mode="mirror")
```

Also available as `result.plot_spectrum(...)`. Returns the matplotlib axes. The area of every band equals its intensity in km/mol.

Parameter	Default	Controls
`result`	required	A VPT2Result or a list of VPT2Transition.
`shape`	`"lorentzian"`	Line shape, `"lorentzian"` or `"gaussian"`.
`fwhm`	None	Full width at half maximum (cm⁻¹). A number, or a dictionary per band type, e.g. `{"fundamental": 8, "overtone": 25}`. None uses 10 (Lorentzian) or 20 (Gaussian).
`kinds`	None	Restrict to band types, e.g. `["overtone", "combination"]`.
`xmin`	None	Lower wavenumber limit.
`xmax`	None	Upper wavenumber limit.
`resolution`	0.5	Grid spacing (cm⁻¹).
`sticks`	True	Draw a stick spectrum.
`by_kind`	True	Draw fundamentals, overtones and combinations as separate dashed curves.
`log`	False	Logarithmic intensity axis, useful in the near-IR.
`normalize`	False	Scale the spectrum to a maximum of 1.
`show_harmonic`	False	Overlay the harmonic spectrum.
`x_unit`	`"cm-1"`	`"cm-1"` or `"nm"`.
`invert_x`	True	Decreasing wavenumbers from left to right (usual for IR).
`ax`	None	Existing matplotlib axes. None creates a new figure.
`label`	None	Legend label of the spectrum.
`color`	`"black"`	Colour of the spectrum.
`reference`	None	Reference bands, e.g. experiment, as `{"positions": [...], "intensities": [...], "label": "exp."}`.
`reference_mode`	`"mirror"`	`"mirror"` draws the reference downwards, `"overlay"` on top of the spectrum.

`broaden_transitions`

```
from overtone import broaden_transitions
broaden_transitions(transitions, x=None, shape="lorentzian", fwhm=None, kinds=None,
                    xmin=None, xmax=None, resolution=0.5, use_harmonic=False)
```

Returns the broadened spectrum as arrays `(x, y)` with y in km mol⁻¹ per cm⁻¹. Also available as `result.spectrum(...)`. The parameters are those of `plot_spectrum`, plus

Parameter	Default	Controls
`transitions`	required	List of `VPT2Transition`.
`x`	`None`	Wavenumber grid. `None` builds one from `xmin`, `xmax` and `resolution`.
`use_harmonic`	`False`	Place the bands at their harmonic instead of their anharmonic wavenumbers.

`broaden`, `lorentzian` and `gaussian`

```
from overtone.spectrum import broaden, lorentzian, gaussian, DEFAULT_FWHM
broaden(positions, intensities, x=None, shape="lorentzian", fwhm=None,
        xmin=None, xmax=None, resolution=0.5)
lorentzian(x, x0, fwhm)
gaussian(x, x0, fwhm)
```

`broaden` broadens arbitrary sticks (`positions` in cm⁻¹, `intensities`). `fwhm` may be one value or one value per line. `lorentzian` and `gaussian` return line shapes of unit area centred at `x0`. `DEFAULT_FWHM` holds the default widths `{"lorentzian": 10.0, "gaussian": 20.0}`.

---

11. Coordinate transformations

`CoordinateTransformer`

```
CoordinateTransformer(freq, modes, masses)
```

Parameter	Controls
`freq`	Harmonic frequencies (cm⁻¹).
`modes`	Normal modes (mass-weighted, one per column).
`masses`	Square roots of the atomic masses, repeated for x, y and z.

Method	Transformation
`cart_to_norm(R)`	Cartesian displacements (Å) to mass-weighted normal coordinates Q.
`norm_to_cart(Q)`	Normal coordinates Q to Cartesian displacements.
`norm_to_reduced_norm(Q)`	Q to dimensionless coordinates q.
`reduced_norm_to_norm(q)`	q to Q.
`cart_to_reduced_norm(R)`	Cartesian displacements to q.
`reduced_norm_to_cart(q)`	q to Cartesian displacements.

---

12. Engines

Engines decide how the single points are run. They are passed to `calculate_anharmonics` with the parameter `engine`.

SerialEngine

```
from overtone.engines import SerialEngine
SerialEngine(restart=True, position_tolerance=1e-08)
```

Runs all single points one after another in the current process. This is the default engine of `calculate_anharmonics`.

Parameter	Default	Controls
`restart`	`True`	Write every finished structure immediately to `calculations/job_<index>.traj` in the output directory, and reuse finished structures of an earlier run.
`position_tolerance`	`1e-08`	Largest position difference (Å) for which a saved structure counts as the same structure.

An interrupted run is restarted by simply starting it again with the same directory. Only the missing structures are calculated. A saved structure is reused only if the atoms and positions match and all requested properties (e.g. forces or dipoles) are present. Progress is printed to stderr, one line per calculated structure, so it also appears in cluster log files.

After each structure, the results are frozen. A calculator shared between structures (such as an on-the-fly learning model) is therefore called exactly once per structure.

ShephexEngine

```
from overtone.engines import ShephexEngine
from shephex.executor import SlurmExecutor
ShephexEngine(executor=None)
```

Distributes the single points with Shephex. `executor=None` runs them locally. A `SlurmExecutor` submits them to SLURM. Interrupted runs can be resumed, because finished single points are not repeated. Requires `pip install "overtone[cluster]"`.

calculate_trajectory

Both engines provide

```
calculate_trajectory(trajectory, setup_calculator, properties=("energy",), ...)
```

Parameter	Default	Controls
`trajectory`	required	Structures to calculate.
`setup_calculator`	required	Function that returns a calculator.
`properties`	`("energy",)`	ASE properties to evaluate, e.g. `"forces"` or `"dipole"`.
`trajectory_path`	`None`	`ShephexEngine` only. File for the input structures.
`output_directory`	`None`	Directory for the individual calculations. The `SerialEngine` stores finished structures there for restarts.
`overwrite`	`False`	`ShephexEngine` only. Recalculate finished single points.

A custom engine can be written by deriving from `overtone.engines.BaseEngine` and implementing `calculate_trajectory`.

13. Files written by OVERTONE

With `save_results=True`, `calculate_anharmonics` writes

File	Content
`harmonics/harmonics_result.json`	`HarmonicsResult`
`harmonics/harmonic_displacements.traj`	Structures of the harmonic analysis with their results
`anharmonics/derivatives_config.json`	`AnharmonicDerivativesConfig`
`anharmonics/anharmonic_displacements.traj`	Displaced structures with their results
`anharmonics/derivatives_result.json`	`AnharmonicDerivativesResult`
`anharmonics/dipole_derivatives.json`	`DipoleDerivativesResult` (with `intensities=True`)
`anharmonics/vpt2_config.json`	`VPT2Config`
`anharmonics/vpt2_result.json`	`VPT2Result`

Independent of `save_results`, the `SerialEngine` stores every finished single point in `harmonics/calculations/` and `anharmonics/calculations/` as `job_<index>.traj`. These files make interrupted runs restartable.

All JSON files can be read back with the `load` method of the matching class.

14. Recommendations and troubleshooting

Converge tightly. Optimise the structure to fmax $\approx 10^{-4}$ eV/Å or lower. Converge every single point tightly, for example with ORCA `VeryTightSCF` or tblite `accuracy=0.01`. Use a fine integration grid for DFT.

Restarting an interrupted run. Start the same script again with the same directory. Finished single points are reused, and only the missing ones are calculated. If you optimise the structure in your script, save the optimised structure and read it on a restart. A re-optimised structure can differ in the last digits, and then no saved single point would match.

Use forces where possible. `derivative_source="forces"` needs far fewer single points. Check the cost first with `dry_run=True`.

Keep the default step. `anharmonic_delta=0.2` balances numerical noise and truncation error. Smaller steps amplify noise, much larger steps distort soft modes.

Soft modes and torsions. VPT2 is unreliable for very soft modes and nearly free rotations. Exclude them with `exclude_below` (e.g. 100 cm^{-1}) or `active_modes`.

“Active modes with imaginary or very low frequency”. The structure is not a minimum, or constrained atoms leave free rotations. Tighten the optimisation, fix at least three non-collinear atoms, or exclude the modes.

Warning about degenerate modes. Linear molecules and symmetric tops have degenerate modes. OVERTONE uses the VPT2 expressions for non-degenerate modes, so results for these modes are approximate.

Bands marked as resonant. These bands take part in a Fermi resonance. Use `method="gvpt2"` (the default) for quantitative energies and intensities.

Machine-learning and active-learning calculators. Never mix exact and predicted energies within the same finite-difference stencil. Small differences between them act like noise.